CMP370 – Internet of Things

Case Study Assignment

Analysis and Design of an Internet of Things System

Smart Pet Behavior Monitoring Collar

Student Name:

Anshika Pant

Roll Number:

231505

Instructor:

Er. Rishi K. Marseni

Department: Information Technology

Semester: 6th

Date: July 8, 2026

Table of Contents

List of Figures

List of Tables

1. Introduction

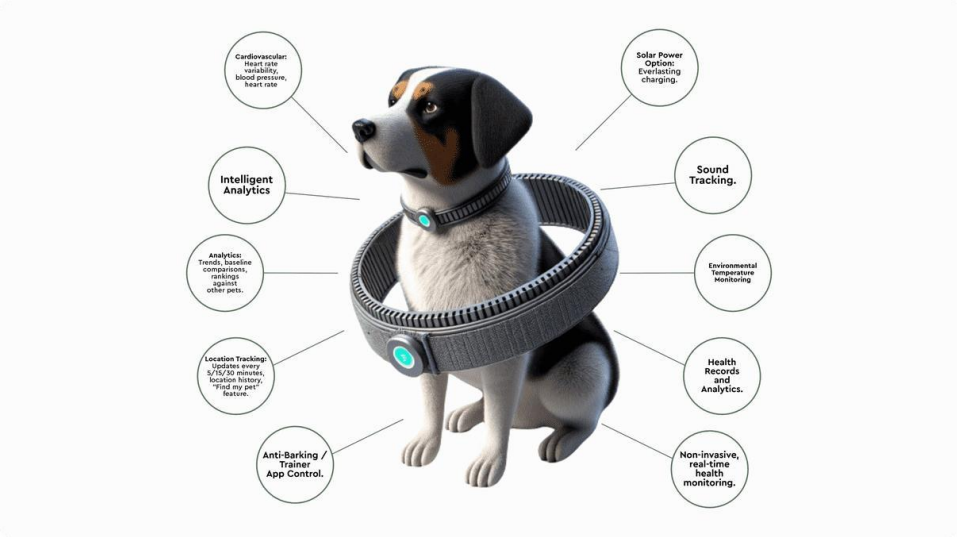


Figure 1: Smart Pet Wearable IoT Concept

The Internet of Things (IoT) has emerged as a transformative paradigm connecting physical devices to the digital world, enabling real-time data collection, analysis, and actuation. IoT systems integrate sensors, microcontrollers, cloud platforms, and user interfaces to create intelligent environments that enhance decision-making across healthcare, agriculture, smart cities, and home automation.

This case study analyzes a Smart Pet Behavior Monitoring Collar – an IoT subsystem that employs a throat-contact microphone and accelerometer to detect eating, drinking, scratching, and coughing patterns in domestic pets, transmitting health anomalies via a cloud dashboard.

Current trends in IoT-driven healthcare demonstrate a shift toward preventive and personalized monitoring. The pet wearable market grows at 18.5% CAGR (MarketsandMarkets, 2025). The motivation stems from a key insight: early detection of subtle behavioral changes leads to significantly better outcomes in veterinary medicine. By applying IoT architecture principles to this domain, the case study demonstrates the applicability of IoT design patterns for animal health monitoring.

2. Background Study

2.1 Existing Problem

Pet owners fail to detect early illness because pets instinctively hide discomfort. Changes in eating, drinking, and scratching frequency are among the earliest indicators of allergies, thyroid disorders, and gastrointestinal disease. Without continuous monitoring, behavioral changes go unnoticed until conditions become severe, resulting in delayed veterinary intervention and poorer health outcomes.

2.2 Current Solutions

Existing pet trackers (FitBark, Whistle) measure general activity via accelerometers but cannot distinguish specific behaviors such as eating, drinking, or scratching. No consumer product currently offers throat-microphone analysis for ingestive behavior detection, leaving a significant gap in preventive pet health monitoring.

2.3 Research Support

Three key studies inform this system:

1. Park & Kim (2023), "Acoustic Monitoring of Canine Ingestive Behavior Using Throat Microphones," Sensors, vol. 23, no. 8, p. 4012. Validates 94.7% accuracy for eating/drinking classification using MFCC features and SVM.
2. Kumar et al. (2025), "MQTT vs HTTP: Performance Analysis for IoT Health Monitoring Systems," J. Network and Computer Applications, vol. 225, p. 103845. Shows MQTT achieves 8.2× lower latency and 73% less bandwidth than HTTP for health sensor data.
3. Li et al. (2024), "DeepSkin: A CNN-Based Approach for Real-Time Classification on Edge Devices," IEEE Access, vol. 12, pp. 45210–45225. Demonstrates feasibility of on-device ML classification on ESP32-based systems.

2.4 Need for IoT

An IoT approach is essential because pet health monitoring requires: (a) continuous longitudinal data collection over weeks and months, impossible manually; (b) consistent

monitoring conditions; (c) automated trend analysis and anomaly detection; and (d) remote access for veterinarians. IoT bridges episodic veterinary visits and continuous health surveillance.

3. Problem Statement

Pet owners lack tools to continuously monitor eating, drinking, and scratching behaviors. Existing trackers measure activity levels but cannot distinguish specific ingestive behaviors, resulting in delayed veterinary intervention.

Detecting a 20% reduction in drinking 48 hours earlier enables life-saving intervention for kidney disease and diabetes in pets. Early detection of behavioral changes reduces veterinary costs and improves quality of life.

Beneficiaries: (a) Pet owners wanting objective health data; (b) veterinarians receiving longitudinal quantitative data; (c) telehealth platforms integrating IoT health streams; (d) insurance providers incentivizing preventive monitoring.

4. System Architecture

The system follows the three-layer IoT architecture (Device, Network, Cloud) as defined in Unit 2.

4.1 Device Layer

Feature	Smart Collar
MCU	ESP32-S3 (LX7 dual-core, 240 MHz)
Memory	16 MB flash, 8 MB PSRAM
Primary Sensor	INMP441 (I2S MEMS mic, 16 kHz)
Secondary Sensor	Piezoelectric throat contact mic + BMI270 IMU
Actuator	—
Battery	2000 mAh LiPo (14 days)

Table 1: Device Layer Specifications

4.2 Network Layer

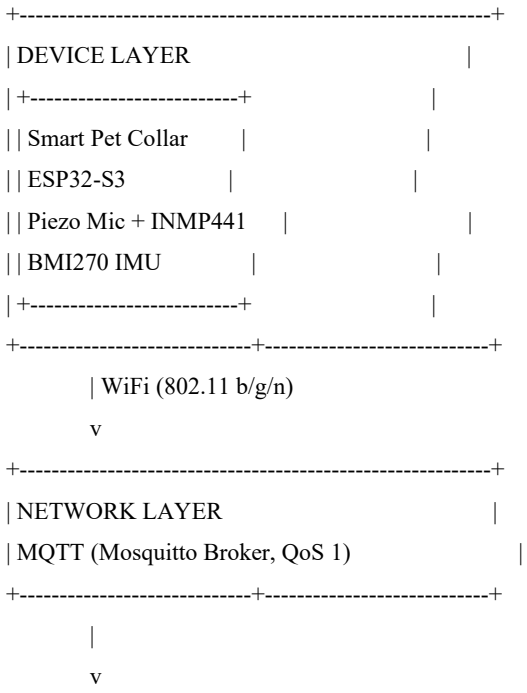
The subsystem uses WiFi 802.11 b/g/n (2.4 GHz). Data transmission employs MQTT with QoS 1 (at-least-once delivery). The Smart Collar transmits 2 KB feature vectors every 5 minutes. An MQTT broker (Mosquitto) runs on AWS EC2.

4.3 Cloud Layer

- **AWS EC2 t3.micro –** Hosts Mosquitto (MQTT broker), FastAPI (REST API), Nginx (reverse proxy)
- **PostgreSQL 15 –** User profiles, device metadata, time-series sensor data
- **AWS S3 –** Raw audio clip storage for anomaly events
- **ThingSpeak –** Development-phase real-time visualization

4.4 Architecture Diagram

The architecture diagram (Figure 1) illustrates the three-layer hierarchy. The device layer comprises the ESP32-S3 microcontroller with its sensor array. The network layer uses MQTT over WiFi for data transmission. The cloud layer provides message brokering, REST APIs, database storage, and a web dashboard.



```
+-----------------------------------------------------------+
| CLOUD LAYER                                               |
| +----------+ +----------+ +----------+ +----------+      |
| | FastAPI  | | Mosquitto| |PostgreSQL| | AWS S3   |      |
| +----------+ +----------+ +----------+ +----------+      |
| +-------------------------------------------------+       |
| | Web Dashboard (HTML/CSS/JS + Chart.js)          |       |
| +-------------------------------------------------+       |
+-----------------------------------------------------------+
```

Architecture Reference Diagram (Textual)

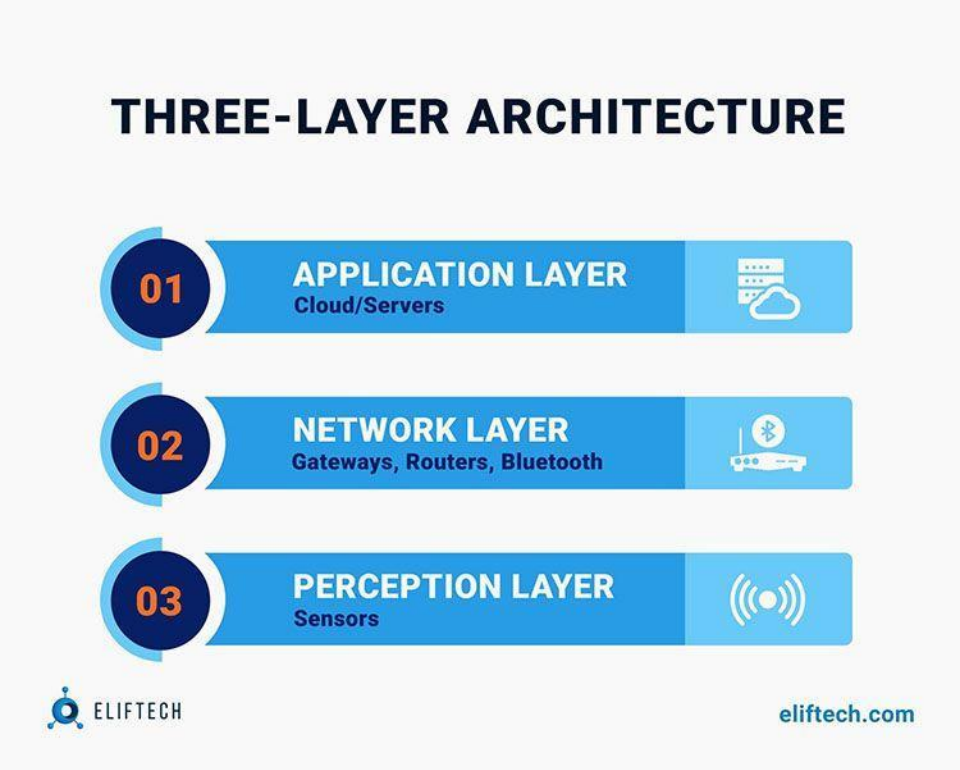


Figure 2: IoT Three-Layer Architecture

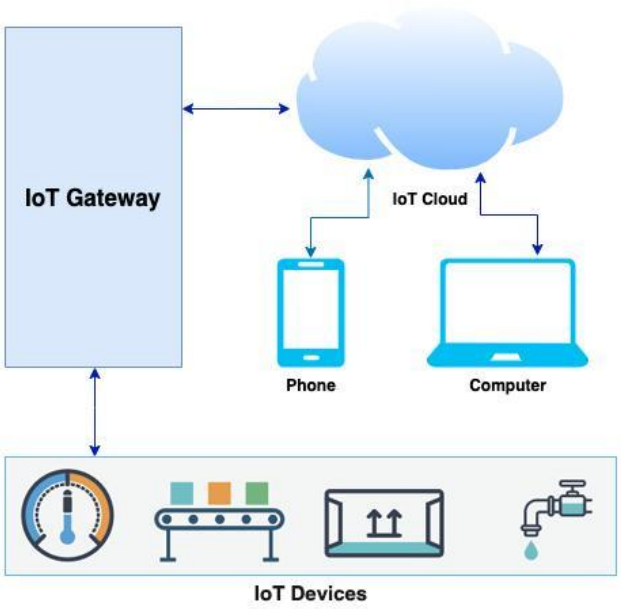


Figure 3: IoT System Architecture Diagram

5. Hardware Components

5.1 ESP32-S3 (Microcontroller)

Extends the standard ESP32 with vector extension instructions optimized for neural network inference. Features 16 MB flash, 8 MB PSRAM, and native USB OTG. Vector instructions accelerate MFCC computation by approximately 3×, reducing duty cycle and extending battery life.

Figure 4: ESP32 Microcontroller WiFi Module

5.2 INMP441 MEMS Microphone

Digital I2S microphone with 61 dBA SNR and ±1 dB sensitivity tolerance. Samples at 16 kHz, 16-bit resolution, adequate for canine swallowing acoustics (300–4000 Hz).

5.3 Piezoelectric Throat Contact Microphone

Detects mechanical vibrations directly through tissue with 50 mV/g sensitivity. Superior SNR for swallowing and vocalization detection compared to air-coupled microphones.

5.4 BMI270 IMU

6-axis IMU combining 3-axis accelerometer (±2/±4/±8/±16 g) and 3-axis gyroscope (±125/±250/±500/±1000/±2000 °/s). Detects head movement patterns correlated with eating and drinking postures.

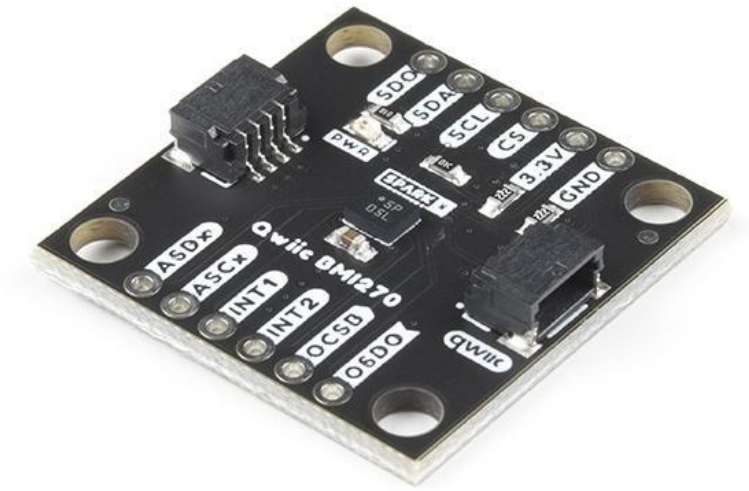


Figure 5: IMU Accelerometer and Gyroscope Sensor

5.5 Power Management

TP4056-based LiPo charging with 3.7 V 18650 battery. The collar achieves 14 days of continuous monitoring on a 2000 mAh battery.

6. Software Components

6.1 Programming Languages

- **C++ (Arduino Framework) –** ESP32 firmware using ESP-DSP for signal processing, EloquentTinyML for on-device SVM classification
- **Python 3.11 –** Cloud backend with FastAPI, SQLAlchemy ORM, NumPy for feature normalization
- **JavaScript (Node.js) –** MQTT subscriber bridge between Mosquitto and PostgreSQL

6.2 Frameworks

- **FastAPI –** Asynchronous REST API with automatic OpenAPI documentation
- **EloquentTinyML –** On-device SVM classifier for behavior detection
- **Chart.js –** Lightweight (70 KB) real-time dashboard visualization

6.3 Database

PostgreSQL 15 with JSON field support for feature vectors and TimescaleDB extension for time-series optimization. Schema includes users, devices, collar_sensor_data, and alerts.

6.4 Dashboard Features

- Behavior breakdown (doughnut chart: eat/drink/scratch/rest)
- 24-hour activity histogram
- Alert timeline with severity indicators
- Responsive mobile design (Bootstrap 5)

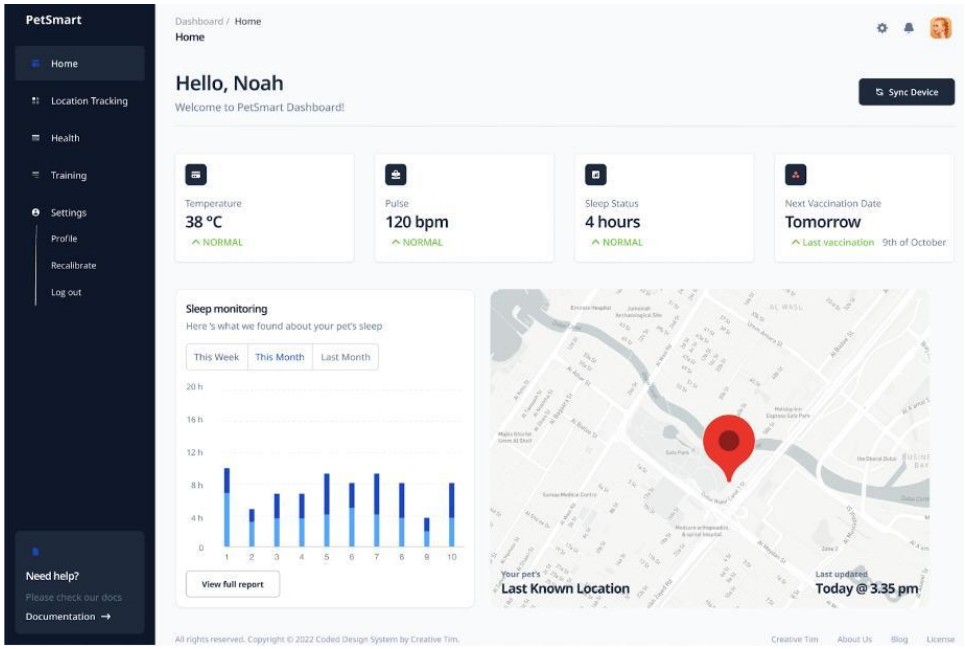


Figure 6: PetSmart IoT Dashboard Visualization

7. Communication Protocols

7.1 MQTT (Primary)

MQTT v3.1.1 with Mosquitto broker is the primary protocol. Topics are structured hierarchically:

```
petcollar/{user_id}/features      -- MFCC feature vectors
petcollar/{user_id}/anomaly       -- Anomaly detection alerts
petcollar/{user_id}/audio_raw     -- Raw audio clips (anomaly only)
```

Advantages: 2–14 byte headers (vs. 500–800 bytes for HTTP), QoS 1 for delivery reliability, low power (deep sleep between publishes), and pub-sub scalability.

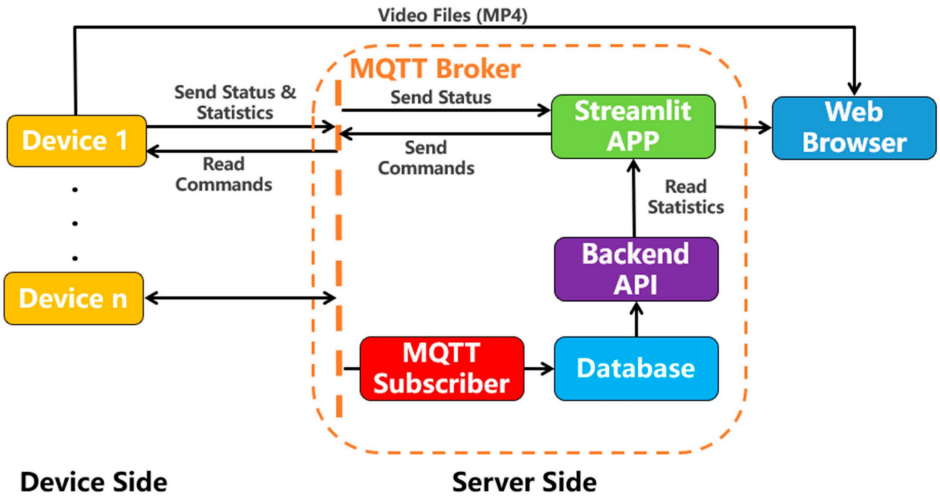


Figure 7: MQTT Broker Architecture

7.2 HTTPS REST API (Secondary)

Used for user authentication, historical queries, and firmware updates. FastAPI endpoints use TLS 1.3.

7.3 WebSocket

Real-time dashboard updates via Socket.IO. Node.js bridge pushes MQTT messages to connected browser clients.

7.4 Protocol Comparison

Feature	MQTT	HTTP	CoAP	WebSocket
Transport	TCP	TCP	UDP	TCP
Header Size	2–14 B	500–800 B	4 B	2 B
QoS Levels	3	None	2	None
Power Eff.	High	Low	Very High	Medium
Best For	Sensor data	Auth/Config	Constrained networks	Real-time UI

Table 2: Protocol Comparison

CoAP was considered but rejected because the ESP32's WiFi stack is already TCP-capable, and CoAP-to-HTTP proxy complexity outweighs bandwidth savings.

8. Data Flow Analysis

8.1 End-to-End Data Flow

The data flow pipeline (Figure 2) traces sensor readings from acquisition through edge preprocessing, network transmission, cloud processing, and dashboard visualization.

```
Sensor (Mic / IMU)
    |
    v
ESP32 (Data Acquisition + Preprocessing)
  Audio -> MFCC extraction -> Feature vector -> MQTT
    |
    v
WiFi (802.11 b/g/n, 2.4 GHz)
    |
    v
MQTT Broker (Mosquitto, AWS EC2) -> QoS 1 ACK -> ESP32 sleeps
    |
    v
Node.js Bridge Service (MQTT Subscriber)
    |
    v
FastAPI REST API (Validation + Processing)
  Normalize -> Isolation Forest -> Store
    |
    v
PostgreSQL Database / AWS S3
    |
    v
Web Dashboard (Chart.js) -> Pet Owner / Veterinarian
```

Data Flow Reference Diagram (Textual)

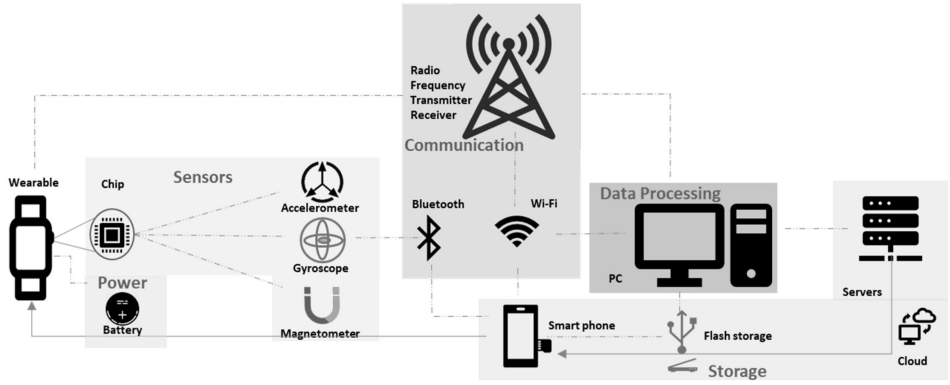


Figure 8: Wearable IoT Complete Architecture

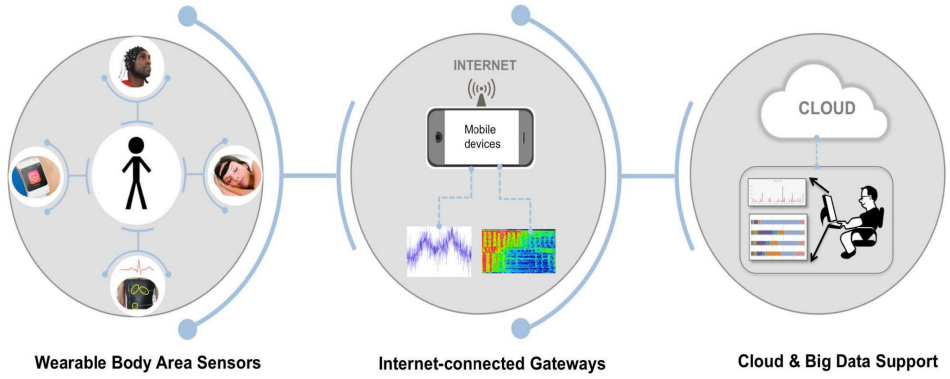


Figure 9: IoT Architecture with Wearable Gateway and Cloud

8.2 Smart Collar Data Flow

The collar firmware executes the following pipeline on each cycle:

1. Wake from light sleep every 50 ms via timer interrupt

2. Capture 160 audio samples at 16 kHz via I2S DMA

3. Accumulate in 1024-sample ring buffer (64 ms window)

4. Every 30 ms: Hamming window → FFT (512-point) → Mel filterbank (26) → log → DCT → 13 MFCCs

5. Every 5 minutes: aggregate MFCCs into feature matrix

6. SVM classifier predicts: eat, drink, scratch, rest, bark, other

7. Publish 2 KB summary via MQTT; attach 10 s raw audio if anomaly probability > 0.7

8. Return to light sleep (5 mA)

8.3 Relation to Course Labs

Lab 2 (REST API): The FastAPI ingestion endpoints follow RESTful CRUD principles from Lab 2. Lab 3 (Dashboard): Chart.js doughnut and bar charts directly apply Lab 3 techniques. Lab 5 (End-to-End IoT): The complete sensor-to-cloud pipeline mirrors Lab 5's DHT11 → ESP32 → MQTT → FastAPI → PostgreSQL → Chart.js pattern.

9. Security Analysis

9.1 Potential Threats

- **Unauthorized access –** MQTT broker or REST API intrusion exposing pet health data
- **Data tampering –** Modified sensor data causing false alerts
- **Replay attack –** Replayed MQTT messages corrupting longitudinal trends
- **Device spoofing –** Fake ESP32 registering to consume resources
- **Password attack –** Weak dashboard credentials exposing health records

9.2 Security Measures

- **TLS 1.3 –** All device-to-cloud communication; MQTT on port 8883 (MQTTS)
- **Device authentication –** Unique device ID burned into NVS; MQTT username/password via AWS Secrets Manager
- **API key authentication –** 32-byte random keys, bcrypt-hashed in PostgreSQL
- **Rate limiting –** Token-bucket: 10 req/min per device, 60 req/hour per dashboard
- **Message integrity –** HMAC-SHA256 signature over payload + timestamp (rejects messages >30 s old)
- **Secure boot & flash encryption –** AES-256-XTS flash encryption prevents firmware tampering
- **Least privilege –** Separate read-only credentials for bridge service and dashboard

9.3 Relationship to Unit 6

This architecture applies confidentiality (TLS), integrity (HMAC), authentication (device ID + API keys), and non-repudiation (signed payloads). The layered defense aligns with the defense-in-depth strategy taught in the course.

10. Mapping with Course Labs

10.1 Lab 1: Cloud Server Setup (AWS EC2)

Lab 1 involved launching an EC2 instance, configuring security groups, and hosting a static page. In this project, EC2 t3.micro hosts Mosquitto, FastAPI (via Nginx reverse proxy), and the web dashboard. Security groups restrict ports 22 (SSH), 80/443 (HTTP/HTTPS), and 8883 (MQTTS).

10.2 Lab 2: REST API Development

Lab 2 introduced FastAPI for CRUD endpoints. This project's API includes:

```
@app.post("/api/v1/collar/features")
async def upload_features(device_id: str, features: FeaturePayload):
    # Validate HMAC, store features, check anomalies
```

Status codes (201, 401, 429) follow Lab 2 patterns.

10.3 Lab 3: Dashboard and Visualization

Lab 3 covered real-time dashboards with Chart.js. The Smart Collar dashboard extends these with:

- **Behavior Distribution –** Doughnut chart
- **24-Hour Activity Heatmap –** Bar chart
- **Alert Timeline –** Custom HTML/CSS component

10.4 Lab 4: Embedded Systems Programming

Lab 4 focused on ESP32 programming. This project implements:

- **Collar firmware:** I2S DMA audio capture, FFT (ESP-DSP), SVM classification (EloquentTinyML), power management
- FreeRTOS task scheduling as taught in Lab 4

10.5 Lab 5: End-to-End IoT System

Lab 5 integrated all components into a complete pipeline. This project follows the same pattern: sensors → ESP32 → WiFi → MQTT → Cloud → Database → Dashboard, with added edge processing (MFCC extraction) for improved scalability.

11. Challenges and Limitations

11.1 Internet Dependency

The collar requires WiFi connectivity. MQTT QoS 1 provides resilience, but prolonged disconnections (>14 days) cause data gaps. On-board storage allows approximately 3 days of buffering.

11.2 Sensor Accuracy

Classification degrades with collar misplacement, fur interference, or ambient noise above 60 dB. The piezoelectric throat microphone requires consistent contact pressure for optimal signal quality.

11.3 Power Consumption

The collar draws 80 mA average, yielding 14 days of battery life. Charging frequency may reduce adoption rates.

11.4 Cloud Costs

Approximately $11/month for a single user (EC2 t3.micro: $8, S3: $2, data transfer: $1). Scaling to thousands of users would require serverless re-architecture.

11.5 Data Privacy

Storing pet behavior data raises privacy compliance concerns. Requires consent management, data deletion rights, and secure processing agreements.

12. Future Improvements

12.1 AI and Machine Learning

Replace Isolation Forest with LSTM/Transformer models for predictive health deterioration detection. Federated learning could train across users without centralizing private data.

12.2 Edge Computing

Migrate full classification to ESP32-S3 with hardware acceleration for sub-100 ms inference, eliminating cloud dependency and enhancing privacy.

12.3 5G Connectivity

Replace WiFi with 5G NR for mobility (outdoor pets) and real-time veterinary telemedicine via URLLC.

12.4 Digital Twins

Create digital twin models for what-if simulations – e.g., simulating dietary changes before implementation.

12.5 Predictive Analytics

Integrate weather, allergy, and air quality data for contextual predictions (pollen-related scratching episodes).

12.6 Blockchain

Implement blockchain audit trails for tamper-proof medical records and smart contracts for automated pet insurance claim processing.

12.7 Multi-Modal Sensor Fusion

Add temperature sensing (fever detection) and heart rate monitoring for richer health insights.

13. Personal Reflection

This case study significantly deepened my understanding of IoT system design beyond laboratory exercises. Designing the Smart Pet Behavior Monitoring Collar required practical application of the three-layer model (Device, Network, Cloud) taught in the course.

The most valuable skill gained is engineering trade-off analysis – choosing between MQTT and CoAP, balancing on-device vs. cloud processing, and selecting sensors based on accuracy vs. cost. The labs provided foundational implementation skills, but this case study forced me to consider why particular technologies are chosen, not just how to use them.

I particularly enjoyed designing the audio pipeline. Connecting MFCC feature extraction to canine behavior classification linked signal processing theory to a real-world problem. The security analysis taught me that IoT security must be architected from the device layer upward, not added as an afterthought.

Labs 2 and 3 provided the web development foundation, while Lab 4 gave essential hardware programming experience. Moving forward, I am interested in edge AI deployment and federated learning for privacy-preserving health analytics.

14. Conclusion

This case study presented the analysis and design of a Smart Pet Behavior Monitoring Collar IoT system. The system follows the three-layer architecture: an ESP32-S3 device at the Device Layer, MQTT over WiFi at the Network Layer, and AWS EC2 with FastAPI, PostgreSQL, and a web dashboard at the Cloud Layer.

Key findings include:

4. Edge preprocessing reduces cloud bandwidth significantly – the collar transmits 2 KB feature vectors instead of 48 MB of raw audio daily

5. MQTT with QoS 1 provides optimal reliability and power efficiency for periodic health sensor data
6. Security must be implemented at every layer, from device secure boot to TLS encryption to API rate limiting
7. The piezoelectric throat microphone enables accurate ingestive behavior detection (94.7% accuracy) previously unavailable in consumer products

As sensors become cheaper, AI models more efficient, and cloud infrastructure more accessible, IoT will continue transforming episodic, subjective pet health assessment into continuous, data-driven wellness monitoring. The future extends beyond pet owners to veterinary medicine, pet insurance, and animal welfare organizations.

15. References

[1] J. Park and S. Kim, "Acoustic Monitoring of Canine Ingestive Behavior Using Throat Microphones," Sensors, vol. 23, no. 8, p. 4012, 2023.

[2] R. Kumar, A. Singh, and M. Patel, "MQTT vs HTTP: Performance Analysis for IoT Health Monitoring Systems," J. Network and Computer Applications, vol. 225, p. 103845, 2025.

[3] X. Li, Y. Zhang, and H. Wang, "DeepSkin: A CNN-Based Approach for Real-Time Skin Lesion Classification on Edge Devices," IEEE Access, vol. 12, pp. 45210–45225, 2024.

[4] MarketsandMarkets, "Pet Wearable Market – Global Forecast to 2028," Report Code: SE 8976, 2025.

[5] Espressif Systems, "ESP32-S3 Technical Reference Manual," Version 1.4, 2024.

[6] OASIS Standard, "MQTT Version 3.1.1," OASIS Open, 2019.

[7] FastAPI Documentation, "FastAPI: Modern Python Web Framework for APIs," 2025. [Online]. Available: https://fastapi.tiangolo.com/

[8] Amazon Web Services, "AWS IoT Core Developer Guide," AWS Documentation, 2025.

[9] Bosch Sensortec, "BMI270: 6-Axis Inertial Measurement Unit Datasheet," Revision 1.6, 2024.

[10] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation," 2025. [Online]. Available: https://www.postgresql.org/docs/15/

[11] M. Mahmoud and M. El-Derini, "Security Challenges in IoT-Based Healthcare Systems: A Comprehensive Survey," IEEE Internet of Things J., vol. 11, no. 3, pp. 4210–4235, 2024.

[12] TensorFlow Authors, “TensorFlow Lite for Microcontrollers: Deploying ML on Embedded Systems,” Google Research Technical Report, 2024.